

MTH209: Midsem Exam - Question 1

1. Follow the instructions **exactly**.
2. If your code is not properly commented or horizontal/vertical spacing is missing, points will be deducted.
3. You are only allowed to use the libraries that we have discussed in the course, and no other libraries are allowed to be called.

Question

Your goal is to write a function, `svd_compress` that takes an `imager` image, and returns a SVD compressed image. However, the input image will be a colored image, and so the SVD will be done in a particular way. Suppose the number of pixels in the image are $n \times m$. Implement image compression in the following way:

1. Stack the three channels of colors on top of each other to create a $3n \times m$ matrix.
2. Implement SVD with k components, where k is the smallest number of components that explain at least p proportion of the singular values of the stacked image. p will be an input to the function. That is for singular values λ_i :

$$k \text{ is the smallest integer } j \text{ such that } \frac{\sum_{i=1}^j \lambda_i}{\sum_i \lambda_i} > p.$$

3. Unstack the lower-dimensional matrix back to a three channel array. This can be done by creating an empty array using the code below and then filling the appropriate components.
4. Save this unstacked image in an array called `compressed_image`. This array should have the same dimensions as above the input image: $n \times m \times 3$.

Your function should look something like:

```
# img: an imager loaded image. NOT the location of a file on the computer
# p: a proportion. Assume it is between 0 and 1.
svd_compress <- function(img, p)
{

  # extract the image removing time dimension
  img_array <- img[, , 1]
  n <- dim(img_array)[1]
  m <- dim(img_array)[2]

  # stacking the image
  stacked <- rbind(img_array[, , 1],
                  img_array[, , 2],
                  img_array[, , 3])

  # do svd, determine k and compress
```

```

...
# foo is the stacked but compressed image
foo <- ...

# unstack the compressed image
compressed_image <- array(NA, dim = c(n, m, 3))
compressed_image[, , 1] <- foo[1:n, ]
compressed_image[, , 2] <- foo[(n+1):(2*n), ]
compressed_image[, , 3] <- foo[(2*n+1):(3*n), ]

# this return object should be an array of size n x m x 3
return(compressed_image)
}

```

Testing

The image of a dog is provided to help you test your function. Here is a way to test your function:

```

library(imager)
img <- load_image("dog.jpeg")
compressed <- svd_compress(img, p = .90)

```

Submission Instructions

INSTRUCTIONS: copy and paste the final function into a file called `question1.R` and upload it to mooKIT. Upload **ONLY the function** and nothing else. This question has 25 marks.